

Ray tracing on CPU

Bc. Josef Jech
FIT CTU
Prague, Czechia
josefjech.mail@gmail.com

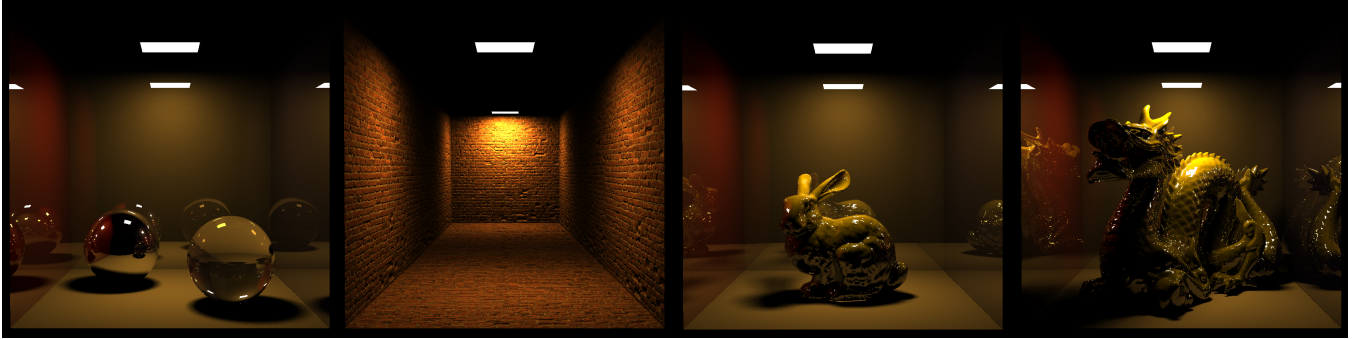


Figure 1: Ray tracer showcase of renders using Phong illumination

CCS Concepts

• **Computing methodologies** → **Ray tracing; Reflectance modeling; Visibility; Texturing.**

1 Introduction

Jako semestrální práci na předmět Počítačová grafika 1 jsem připravil ray tracer v C++, který běží na jednom vlákně CPU. Ray tracing je metoda aproximace globálního osvětlení scény, při které pro každý pixel vystřelíme paprsek a hledáme nejbližší trojúhelník ve scéně, se kterým má paprsek kolizi. Tím pádem není třeba řešit algoritmy na řešení viditelnosti objektů.

Paprsek se následně v bodě průsečíku s trojúhelníkem vyšle do všech světél paprsek, který kontroluje, zda mu něco nestíní v cestě a pokud nestíní, tak spočítá příspěvky světél k barvě pixelu. Spočítá své BRDF a vyšle rekurzivně další paprsky, které simulují odrazy a průsvitnost a jejichž barva se připočítá k barvě pixelu. Pro takovou global illumination metodu existuje mnoho vylepšení a některé zde implementované proberu.

2 Implementation

Ray tracer vysílá primární, stínové, odražené i lomenné paprsky. Umí v základu pracovat s plošnými světly, meshemi ve formátu Obj a materiály ve formátu Mtl. Barvu vyhodnocuje Phongův osvětlovací model. Pomocí uložených UV souřadnic dokáže mapovat textury, jež filtruje bilineární interpolací. Ukládá podrobné statistiky do output/times.log a je konfigurovatelný pomocí config.json.

2.1 Improvements

GGX

Osvětlovací model založený na microfacet BRDF a vylepšující předchozí model Cook-Torrance je používá pro PBR. Je založený na třech členech F,G,D. Ty modelují F - Fresnelův jev, G - Stínování

a maskování microfacet a D - statistická funkce distribuce normál microfacet.

$$f_r(\omega_i, \omega_o) = \frac{D(h) F(\omega_i, h) G(\omega_i, \omega_o)}{4 (\mathbf{n} \cdot \omega_i) (\mathbf{n} \cdot \omega_o)} \quad (1)$$

Pro člen F používá Schlick's approximation, což je rychlejší výpočet fresnelovy rovnice. Pro člen G ve variantě Smith používá funkci Schlick-GGX G1:

$$G_{\text{Smith}}(\omega_i, \omega_o) = G_1(\omega_i) G_1(\omega_o) \quad (2)$$

$$G_1(\omega) = \frac{\mathbf{n} \cdot \omega}{(\mathbf{n} \cdot \omega)(1 - k) + k} \quad (3)$$

A člen D pomocí GGX distribuce:

$$D_{\text{GGX}}(h) = \frac{\alpha^2}{\pi [(\mathbf{n} \cdot \mathbf{h})^2 (\alpha^2 - 1) + 1]^2} \quad (4)$$

Parametr α reprezentuje umocněnou drsnost povrchu. Tuto hodnotu zde počítáme z Phongova exponentu lesklosti h následovně:

$$\text{roughness} = \sqrt{\frac{2}{h + 2}} \quad (5)$$

$$\alpha = \text{roughness}^2 \quad (6)$$

Pro přepnutí BRDF z Phongu na GGX stačí v konfiguračním souboru dat GGX:true.

Fresnel equation & TIR

Fresnelův člen se v rendereru využívá jako koeficient určující, jaká část dopadajícího světla se od povrchu odrazí v závislosti na úhlu pohledu. U transparentních materiálů tento koeficient řídí distribuci energie mezi odraženým a lomeným paprskem. U neprůhledných materiálů pak definuje intenzitu zrcadlového odlesku. Pro efektivní výpočet tohoto jevu je implementována Schlickova aproximace:

$$F(\omega_i, h) = F_0 + (1 - F_0) (1 - (\omega_i \cdot \mathbf{h}))^5 \quad (7)$$

Pokud však pro odražený paprsek nastane situace, kdy se lomenný paprsek nedostane na povrch a zacyklí se, fresnel zajistí, že se všechno světlo od povrchu odrazí.

Mipmapping

Mipmapping je tvorba kaskády(levelů) textur, kde má každý další level o polovinu kratší strany, které se dělí až dokud se nevytvoří level s 1 texelem. Zabráňuje aliasingu textur. Pro filtraci mipmap je potřeba interpolovat texely z bilineárně interpolovaného předchozího a následujícího levelu textury, který jsem vypočítali.

Místo metody pro zjištění levelu založené na derivaci sousedních pixelů jsem použil "footprint estimation" metodu. Tato metoda popisuje kolik pixelů v dané vzdálenosti by se vešlo do jednoho pixelu obrazovky s texturou levelu 0. K získání levelu se spočítá log *ratio*, k tomu jsou potřeba všechny tyto parametry:

- texScale - jak moc je textura nahuštěná na ploše trojúhelníku
- pixelSpreadAngle - zorný úhel jednoho pixelu
- footprintWorld - velikost trojúhelníku na jeden pixel (objekty v dálce větší v poměru k pixelu)
- ratio - kolik texelů z textury level 0 odpovídá jednomu pixelu na monitoru

Normal mapping

Výpočet normál se dělá transformací z normalové mapy v tangent space, který je definovaný původní normálou, tangentou a bitangentou. V každém bodě je uložena normála a tangenta k ní získaná z Gram-Schmidtovy ortogonalizace.

Pak je třeba vytvořit TBN matrix pro kontrétní bod. Normála N a tangenta T interpolovaná pro smooth shading mezi vrcholy trojúhelníka. Problém je v tom, že se interpolaci ztratí mezi N a T ortonormalita a je potřeba znova ortonormalizovat. Na základě N a T je vytvořena bitangenta B a pak stačí normálou z mapy přenásobit TBN maticí.

Gamma correction & Tone Mapping

Protože nejsou změny v intenzitě vnímány lineárně a s barvami se tak počítá, je potřeba nejdříve provést lineární operace s barvami a výsledný pixel umocnit na 1/2.2. Textury jsou většinou u s aplikovanou gamma korekcí, takže ty je potřeba nejdříve umocnit na 2.2.

Ještě před gamma korekcí je možné aplikovat Reinhard Tone Mapping, který lépe distribuuje barvy z HDR do výsledného rozsahu 0-1. Aplikované na GGX, pokud bychom místo funkce postProcessPixel použili gammaClamp.

Druhou možností je aproximace ACES barevného prostoru pomocí filmic křivky, která je u GGX implementovaná jako postProcessPixel a můžeme pro ní měnit expozici. Ref:<https://knarkowicz.wordpress.com/2016/01/06/aces-filmic-tone-mapping-curve/>.

3 Kd-tree

Pro zrychlení hledání průsečíku s trojúhelníkem se používají akcelerační struktury a já jsem implementoval Kd strom. Jedná se o BVS, který dělí prostor v uzlech střídáním dělicí roviny podle osy x,y,z. Popíšu naivní přístup stavbu stromu s dělením prostoru pomocí mediánu a pak pomocí SAH.

3.1 Median Build

Základní (naivní) stavba stromu probíhá rekurzivně metodou rozdělení v mediánu. Dělicí osa (X, Y nebo Z) se v každém uzlu pravidelně střídá v závislosti na aktuální hloubce zanoření ve stromu (typicky jako *hloubka* mod 3).

K určení přesné pozice dělicí roviny se následně využijí těžiště (střed) jednotlivých trojúhelníků. Trojúhelníky se podle souřadnic svých těžišť na aktuálně zvolené ose seřadí a vybere se prostřední index (medián), čímž se geometrie rozdělí na dvě poloviny pro levý a pravý podstrom. Rekursivní dělení se ukončí a vytvoří se listový uzel ve třech případech: pokud je dosaženo maximální hloubky stromu, pokud počet trojúhelníků v uzlu klesne pod povolené minimum, nebo pokud se detekuje, že se trojúhelníky nahromadily a rozdělením by žádné neubýly.

3.2 SAH build

Vylepšená metoda využívající Surface Area Heuristic (SAH) se snaží minimalizovat výpočetní cenu budoucího průchodu stromem. Pro každý uzel analyzuje dělení podél všech tří os. Jako kandidáti na dělicí rovinu jsou v implementaci využity hranice (minimum a maximum) AABB jednotlivých trojúhelníků.

Pro každého kandidáta se spočítá, kolik trojúhelníků by spadlo do levého a pravého potomka, a odhadne se cena dělení (*C*) podle vzorce:

$$C = C_{trav} + \frac{S_L}{S_{node}} N_L C_{isect} + \frac{S_R}{S_{node}} N_R C_{isect} \quad (8)$$

kde C_{trav} je konstantní cena průchodu vnitřním uzlem, C_{isect} je cena pro výpočet průsečíku s trojúhelníkem. Zlomek reprezentuje poměr povrchu levého/pravého uzlu ($S_{L/R}$) vůči povrchu celého aktuálního uzlu (S_{node}) a $N_{L/R}$ udává počet trojúhelníků v dané větvi. Následně je vybrána dělicí rovina s nejmenší cenou. Pokud je ale tato minimální cena stále vyšší než samotné vytvoření listu (cena odpovídající pouze průsečíkům: $N \cdot C_{isect}$), k dělení nedojde a uzel se stane listem.

3.3 Traversal

Průchod stromem je implementován iterativně s využitím předalokovaného zásobníku. V prvním kroku se otestuje průsečík paprsku s hlavním AABB celého stromu, čímž se získají vzdálenosti vstupu paprsku do stromu (t_{min}) a výstupu (t_{max}).

Během průchodu se u každého vnitřního uzlu vypočítá vzdálenost průsečíku paprsku s dělicí rovinou (t_{split}). Na základě směru paprsku algoritmus vyhodnotí, který potomek je bližší a který vzdálenější, a řeší tři situace:

- Pokud paprsek protíná dělicí rovinu až za výstupním bodem ($t_{max} \leq t_{split}$), pokračuje se pouze do bližšího uzlu.
- Pokud paprsek protíná dělicí rovinu ještě před vstupním bodem ($t_{split} \leq t_{min}$), pokračuje se rovnou do vzdálenějšího uzlu.
- Pokud paprsek prochází oběma uzly, vzdálenější uzel se uloží na zásobník (společně se zkrácenými limity t_{split} a t_{max}) a paprsek pokračuje nejdříve do bližšího uzlu.

Při dosažení listového uzlu proběhne iterace přes pole uložených indexů a výpočet průsečíků se všemi přítomnými trojúhelníky. Díky

Table 1: Ray tracer performance results

Scene/Measures (times in ms)	Render time	kRays/s	Build time	#Nodes	Avg tree depth	Avg tris/leaf	mem [KB]	Avg Incid. trav.	Avg ray trav. time	Avg Trav. Depth	Time vs BASE
<i>original/ (34 triangles)</i>											
Phong (No tree)	8059.0	4567.32	-	-	-	-	-	-	-	-	1x
Phong (Rec build)	28491.8	1299.03	2.0768	1711	18.2243	5.8768	92.7852	29.1717	0.0003888	0.9315	3.53x
Phong	15401.3	2391.43	0.7250	183	7.38043	6.9130	5.3438	30.8947	0.0002282	1.8284	1.91x
<i>long/ (30 triangles, textures)</i>											
Phong (No tree)	18362.6	3999.95	-	-	-	-	-	-	-	-	1x
Phong (Rec build)	58295.0	1259.98	0.0860	869	28.9287	3.6966	39.7344	14.6188	0.0003336	0.9140	3.17x
Phong	34366.6	2138.11	0.0878	73	5.7027	7.2973	2.19531	23.5477	0.0002032	0.9269	1.87x
<i>sphere/ (2188 triangles)</i>											
Phong (No tree)	1259320	55.0102	-	-	-	-	-	-	-	-	1x
Phong (Rec build)	178537.0	388.04	0.4661	625	28.8403	10.2460	44.6016	335.3833	0.0054411	0.5451	0.142x
GGX (Rec build)	178611.5	387.91	0.4359	625	28.8403	10.2460	44.6016	335.3750	0.0054054	0.5451	0.142x
Phong	27181.6	2555.70	39.6324	32467	19.2587	6.16219	898.066	18.4960	0.0008414	1.9375	0.022x
GGX	28631.7	2422.02	39.5324	32467	19.2587	6.16219	898.066	18.4959	0.0009029	1.9375	0.023x
Phong - NOTEX (mirror backwall)	36838.7	2544.96	42.7561	32467	19.2587	6.16219	898.066	16.3688	0.0009085	1.4390	0.029x
Phong - NOTEX (diffuse backwall)	24499.7	2827.71	45.2617	32467	19.2587	6.16219	898.066	18.4964	0.0007709	1.9375	0.019x
<i>bunny/ (69463 triangles)</i>											
Phong (Rec build)	8623940.0	7.74	30.4188	16694	34.5229	11.8253	1292.89	8357.9700	0.2192440	0.5693	1x
Phong	27121.2	2479.27	2756.5600	1885477	31.5396	4.5087	46064.2	16.9531	0.0004061	1.8893	0.0034x
GGX	25818.8	2586.38	1878.1200	1885477	31.5396	4.5087	46064.2	16.9516	0.0003987	1.8891	0.0032x
<i>dragon/ (871318 triangles)</i>											
Phong	74322.4	1159.48	25360.1000	19773123	33.2597	5.0403	503607	25.1965	0.0016568	3.1155	1x
GGX	75437.4	1144.00	26826.0000	19773123	33.2597	5.0403	503607	25.1945	0.0017115	3.1155	1.02x

řazení průchodu od nejbližšího po nejvzdálenější lze prohledávání často úspěšně ukončit hned v prvním zasaženém listu.

4 Measurement results

Měření bylo provedeno na scénách Original, long, Sphere(with texture), Sphere(mirror backwall) a výsledné rendery jsou vidět na 2, 3. Renderoval jsem obrázky 800x800px, hloubka rekurze 8, počet samplů na světlo 32 a při zapnutí GGX byl použit ACESFilm s exposure 1.25. V tabluce 1 můžeme vidět výsledky měření renderů, všechny časy jsou v ms a výsledek Time vs Base reprezentuje porovnání času vůči scéně bez poizité akcelareční struktury, pokud to šlo. Pokud není uvedeno jinak, byl použit SAH kd-tree.

Pro jednodušší scény (original a long) můžeme vidět, že celkový čas byl rychlejší bez kd-tree a s ním to bylo cca 2-3 pomalejší.

Na scéně s koulema už vidíme, že kd-tree výrazně pomáhá a to s medianovým buildem s dělením podle mediánu pomohl na scéně s texturama byl 7x rychlejší. Pokud jsem použil SAH build, byl 45X rychlejší. Pokud jsem místo textury dal neodrazivou/difúzní zeď bylo zrychlení 52x a odrazivá/zdcadlová zeď byla pomalejší než textura s normal mapou 34x rychlejší.

Pro scénu s králíkem už to nebylo upočitatelné bez kd-tree, ale i tak už medianový build skoro nestačil a proti němu SAH build nabral zrychlení 312.5x.

Dračí scéna nebyla v mých silách upočitatelná bez stromu ani s medianovým buildem. Alespoň je na něm vidět, že zpomalení při použití GGX místo Phongu je 2%.

5 Solved problems

První problém, na který jsem narazil se objevil při přechodu z velké scény s boxama na malou scénu s koulema. Tam jsem musel začít řešit problémy s různými epsilonama, aby se mi aspoň něco vykreslilo. Jeden epsilon na porovnání floatů nestačil. Druhý epsilon na posun vyslaných paprsků od trojúhelníku pomohl, ale skutečně pomohl až epsilon, který zastavil stínový paprsek dostatečně brzy před světlem.

Problém byl také dost s normálovými mapama a mimappingem.

S mipmapama jsem řešil, že jsem to neměl na jaké scéně otestovat a tu co jsem vytvořil skrávala spustu problémů a hodiny předělávání. Kromě toho byl oříšek zjistit metodu pro zjištění mipmap levelu dané textury ve scéně, viz Mipmapping.

U normálových map jsem musel pochopit, kde se berou tangenty a jak s nima pak pracovat a udržet je stále ortonormální k normále.

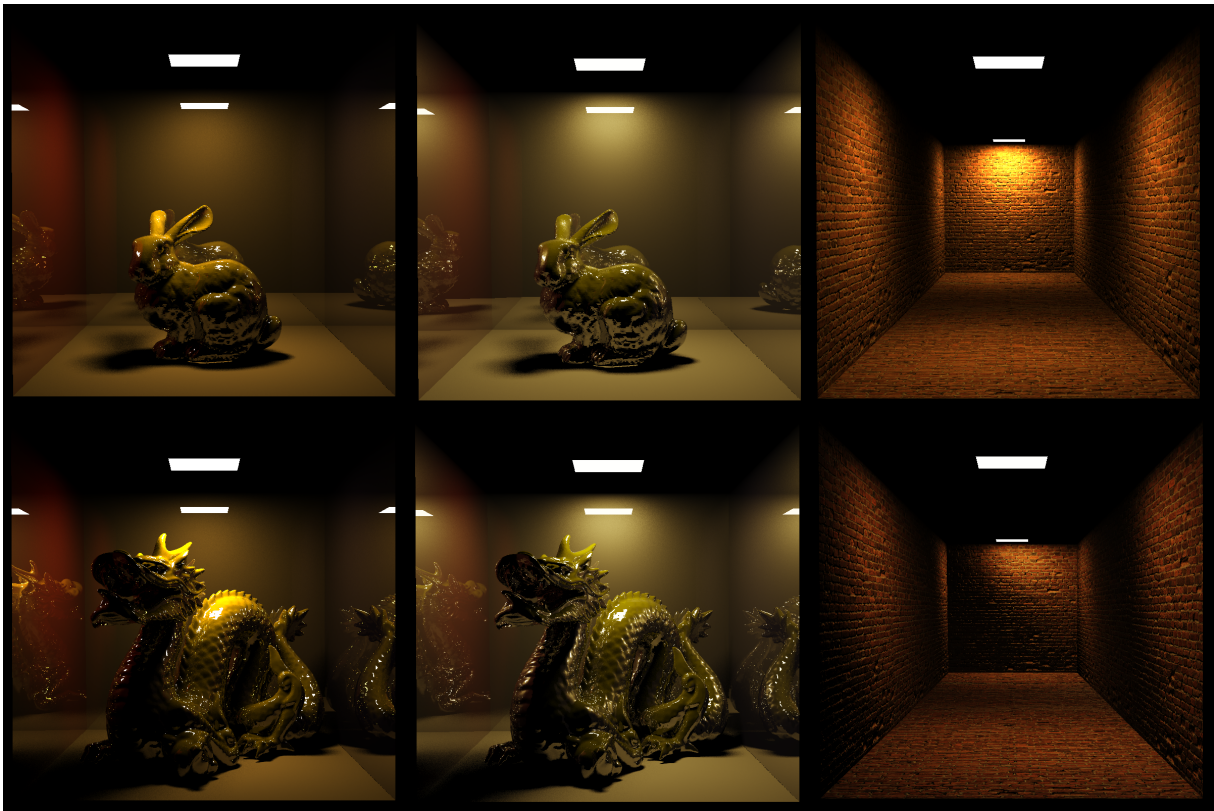


Figure 2: Scenes: Bunny, Dragon, Long

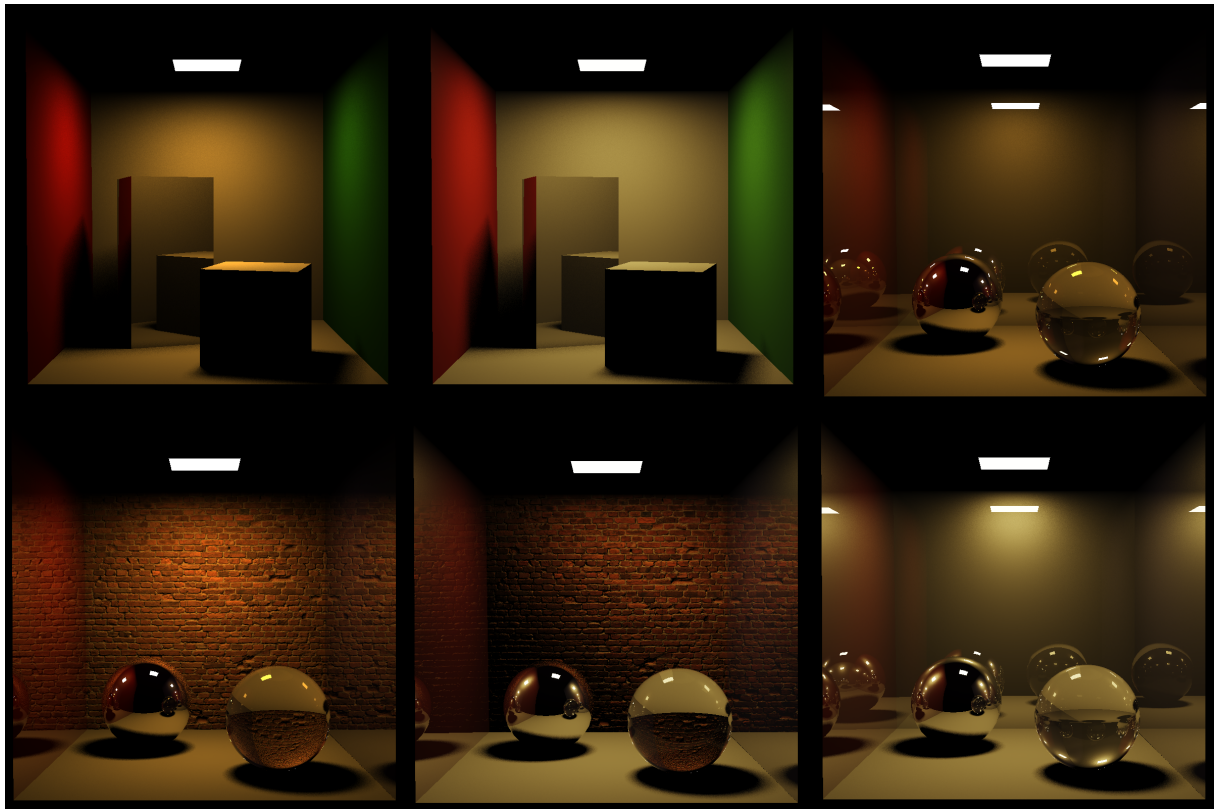


Figure 3: Scenes: Original(Box), Spheres(Textured), Spheres(Mirror backwall)